ICS 604
Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Announcements

- No class on Monday, January 19 (Martin Luther King, Jr. Day)

Lecture 2

- Introduction to Pandas Python Package
  - What are Python modules and packages?
  - What is Pandas?
  - Series and DataFrame classes

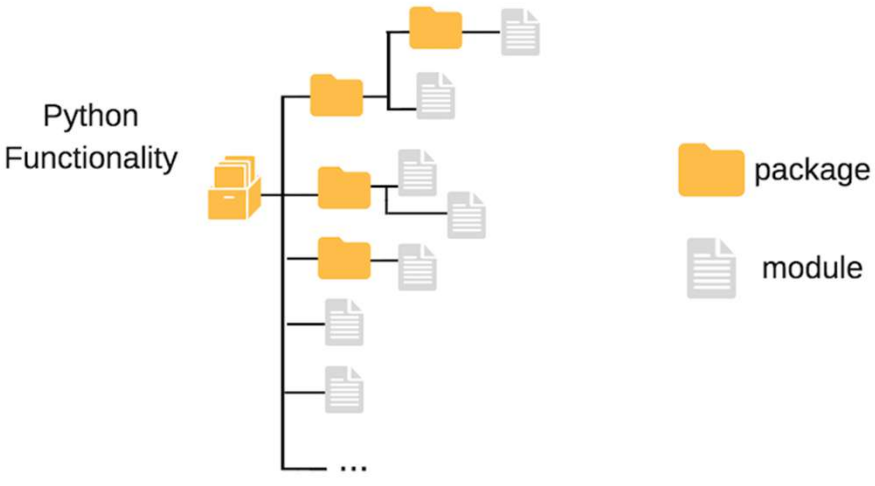
Python modules and packages

- Python implementation of the DRY principle of software engineering
- Module: A module is a single Python file containing definitions and implementations, such as functions, classes, and variables.
  - Used to organize code logically and reuse it across different projects.
    - For example, a file math_operations.py containing various math functions can be a module.

Python modules and packages (cont'd.)

- It's often impractical to combine all the desired functionality into a single module.
- Example: If you work for Hawaiian airlines in analytics, you may separate customer analytics and freight analytics into two different modules.
  - For convenience, you'd place both modules (python files) to live under the same folder.
- A package in Python is merely a directory that contains a collection of modules.
  - It includes an `__init__.py` file to indicate that the directory is a package.
  - Can also contain additional files, such as doc, configuration, etc.
  - A package can further be split into sub-packages.
    - Freight can, for instance, be split into two modules, commercial vs. personal modules etc.
- Top package is often called a library.

Modules and packages



Importing Python modules

- You must import the module using `import` statement to be able to use it.
- We can do this in two ways:
  1. Import **all** the functionality in a module, and with it all the functionality and definitions
     - Keep the functionality bound to module name:

```
>>> import math
```

     - To reference definitions or functionality after this import statement, we use *dot notation*.

Importing Python modules (cont'd.)

- Example:
  - If we want to access the `math` module's definition for π, we would type:

```
>>> math.pi
3.141592653589793
```

  - If we want to compute 4! the `math` module's `factorial` function, we would type:

```
>>> math.factorial(4)
24
```

Importing Python modules (cont'd.)

- You must import the module using `import` statement to be able to use it.
- We can do this in two ways:
  2. Only import some of the module's functionality
     - If we wanted to only import the `math` module's definition for π, we would type:

```
>>> from math import pi
```

     - Then, to reference `pi` all we would type is:

```
>>> pi
3.141592653589793
```

```
>>> pi = 2
>>> print(pi)
2
```

Importing Python modules – Don't

- Do not use the * symbol to import all the functionality. For example:

```
from math import *
```

- The * imports all the functionality into your workspace
  - Functionality can be accessed without using the module name as a prefix. E.g.:

```
r = 2
c = 2 * pi * r
```

- The above:
  - Pollutes your workspace
  - Creates the potential for variable name clashes
  - Makes your code harder to read

Providing a module alias

- It's common to rename (alias) modules when importing them.
  - Saving us some keystrokes when typing the module name
- Example:

```
from matplotlib import pyplot as plt
```

pyplot.scatter( )

- Accessing the functionality and definitions is done by using `plt` instead of `pyplot`

```
plt.scatter(x, y)
```

Providing a module alias (cont'd.)

- When using an alias, the expression "`from X import Y as Z`" is often shortened as "`import X.Y as Z`." For example:

```
import matplotlib.pyplot as plt
```

- Aliases are typically standardized by convention or established by the package developers. For example:
  - Alias for `Pandas` is `pd`
  - Alias for `NumPy` is `np`
  - Alias for `Seaborn` is `sns`
  - Alias for `pyplot` is `plt`
- Import to stick to those conventions.

Types of Python modules and packages

- There are three types of Python modules and packages:
  1. Standard library modules
     - Distributed with Python
     - Do not require installation
     - Must be imported before use
     - Always available once Python is installed
     - Examples: `datetime`, `random`, `collections`, `math`, etc.

Types of Python modules and packages (cont'd.)

  2. Third-party packages commonly included in distributions
     - Not part of the standard library
     - Installed automatically by some Python distributions (e.g., Anaconda)
     - Not available in a minimal Python installation
     - Must be imported before use
     - Examples: `numpy`, `pandas`, `matplotlib`, `scipy`,etc.

Types of Python modules and packages (cont'd.)

3. Third-party packages not included by default
   - Not part of the standard library
   - Must be installed manually (via conda or pip)
   - Often domain-specific or specialized
   - Examples: `biopython`, `astropy`, `tensorflow`, etc.

Example of a module distributed with Python

```
import random

my_list = [1, 2, 3, 4, 5, 6, 7, 8]
print(f"Before the shuffle, my_list is {my_list}.")
random.shuffle(my_list)
print(f"After the shuffle, my_list is {my_list}.")
```

```
Before the shuffle, my_list is [1, 2, 3, 4, 5, 6, 7, 8].
After the shuffle, my_list is [5, 4, 8, 1, 3, 2, 7, 6].
```

Example of third-party packages

```
import pandas as pd
import requests
from io import StringIO    # io is part of the PSL

url = "https://en.wikipedia.org/wiki/List_of_airline_codes"
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
page = StringIO(response.text)

airlines_info = pd.read_html(page, header=0)[0]
airlines_info[airlines_info["Call sign"] == "HAWAIIAN"]
```

	IATA	ICAO	Airline	Call sign	Country/Region	Comments
3116	HA	HAL	Hawaiian Airlines	HAWAIIAN	United States	NaN

Why Pandas?

- Pandas is the de facto package for data wrangling in Python.
- Provides functionality that covers the complete data wrangling workflow.
  - From reading and writing files in multiple formats to handling missing values and to merging datasets.
- Includes basic statistical and visualization functionality to facilitate the analysis.

Simple data wrangling scenario

- Goal: Transform your data to make it more suitable for analysis.
- Example: Suppose you want to display average flight delays for defunct airlines, and test the hypothesis:
  - Do defunct companies have a worse delay track record?

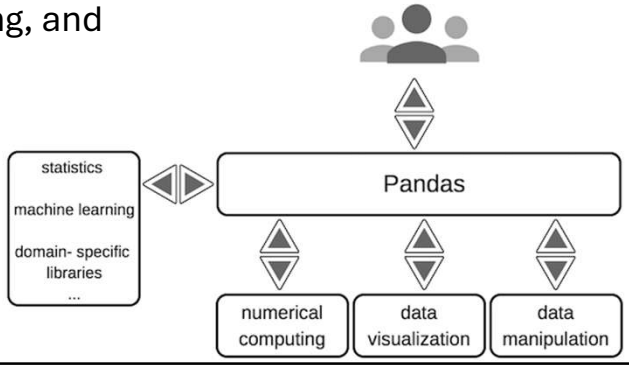
Simple data wrangling scenario: Steps in the process

1. Merge the Wikipedia page listing airline codes with a delays database.
2. Remove entries with missing delay information.
3. Label or group airlines as defunct or active.
4. Normalize delay formats:
   - Some delays are in minutes; others in hours.
   - Delay may be written as 5h20m, other may be 5h:20m or 5:20
   - Ensure consistent format for analysis
5. …

Series and DataFrame Classes

Brief introduction to Pandas

- Pandas is the de facto package for working with tabular data.
- Think of it as Excel on steroids (powerful, but no GUI).
- Supports many file formats (CSV, TSV, Excel, JSON, HDF5, etc.)
- Enables reading, writing, cleaning, and manipulating data efficiently.



Pandas Series and DataFrames

- Pandas relies principally on two types of data structures:
  1. Series: Those are list-like objects that store data in a given order.
     - It helps to think of a Series as a column (or row) in Excel worksheet.
  2. DataFrames: Those are spreadsheet-like tables that contain one or more Series.
     - It helps to think of a DataFrame as a table (or worksheet) in Excel.

About the data

- In the examples, we'll use the cost of drugs prescribed to Medicare patients.
- The complete dataset is publicly available on the Centers for Medicare & Medicaid Services (CMS website):
- This toy dataset contains the following columns:

unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
AB789982	1952310666	Psychiatry	CLONAZEPAM	226	$1,848.88
AV967778	1952310666	Psychiatry	DIAZEPAM	103	$662.87
CC128705	1298765423	Cardiology	NADOLOL	13	
GH890091	1346358827	Family	HYDROCODONE	331	$8,511.14
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	$1,964.49
YY190561	1548247315	Psychiatry	GABAPENTIN	86	$1,807.16
YY572610	1548247315	Psychiatry	MIRTAZAPINE	191	$3,131.96
PL346720	1326175365	Family	OXYCODONE HCL	87	$12,881.04
GZ129032	1518970284	Hemato-oncology	DIGOXIN	54	$3,766.34

About the data (cont'd.)

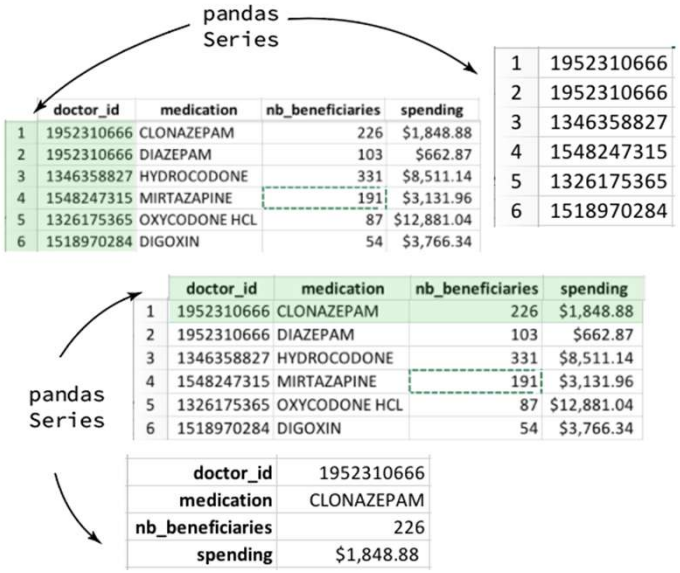
Column	Description
`unique_id`	A unique identifier for a Medicare claim to CMS
`doctor_id`	The unique identifier of the doctor who prescribed the medicine
`specialty`	The specialty of the doctor who prescribed the medicine
`medication`	The medication prescribed
`nb_beneficiaries`	The number of beneficiaries the medicine was prescribed to
`spending`	The total cost of the medicine prescribed

Series

- A Series is conceptually "similar" to a python list.
  - An ordered collection of items.
- All the items in a Series have the **same datatype**.
  - Items are converted to object datatype to maintain this condition.
    - Very common source of bugs or error.
- Each data entry in a Series is associated with a label.
  - Those labels are the index of the data.

Example of Series as rows and columns

- It helps to think of pandas Series as either a column (or a row) of an Excel worksheet.



Creating a pandas Series

- To create a pandas Series, you can call the Series function and pass it a list of values and a list of labels (an index).
  - The index is optional and automatically set to `RangeIndex` (simple range) by default.

```
import pandas as pd
s =  pd.Series(data = [1234, 'DIAZEPAM', 3, '$32'],
               index = ['doctor_id', 'medication', 'nb_beneficiaries', 'spending'])
s
```

```
doctor_id              1234
medication          DIAZEPAM
nb_beneficiaries           3
spending                 $32
dtype: object
```

Creating a pandas Series (cont'd.)

```
import pandas as pd
s = pd.Series(data = [1234, 'DIAZEPAM', 3, '$32'])
s
```

```
0        1234
1    DIAZEPAM
2           3
3         $32
dtype: object
```

```
s.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

Creating a pandas Series (cont'd.)

```
s.index = ["label_1", "label_2", "label_3", "label_4"]
```

```
s.index
```

```
Index(['label_1', 'label_2', 'label_3', 'label_4'], dtype='object')
```

```
s
```

```
label_1        1234
label_2    DIAZEPAM
label_3           3
label_4         $32
dtype: object
```

```
s.values
```

```
array([1234, 'DIAZEPAM', 3, '$32'], dtype=object)
```

Indexing Series

- You can access the data by passing the `iloc` (index location) operator a single index (position) or a list of indexes.

```
s = pd.Series(data = [1234, 'DIAZEPAM', 3, '$32'],
              index = ['doctor_id', 'medication', 'nb_beneficiaries', 'spending'])
s
```

```
doctor_id               1234
medication           DIAZEPAM
nb_beneficiaries            3
spending                  $32
dtype: object
```

```
print(s.iloc[1], s.iloc[-1])
```

```
DIAZEPAM $32
```

Indexing Series (cont'd.)

- You can also access data in a Series by label – either by using the indexing operator with the label, or by passing the `loc` operator a single label or a list of labels.

```
print(s["medication"], s["spending"])
```

```
DIAZEPAM $32
```

```
print(s.loc["medication"], s.loc["spending"])
```

```
DIAZEPAM $32
```

Subsetting Series

- We can index a subset of elements from a Series using a list of indexes.

```
s.iloc[[0, 2, 1]]
```

```
doctor_id               1234
nb_beneficiaries           3
medication          DIAZEPAM
dtype: object
```

- Note that the line above contains two sets of square brackets [[ ]], the first is the indexing operator, the second (inner) is the delimiter for the list.

Subsetting Series (cont'd.)

- Subsetting Series can also be done using slice notation with the range operator (":").

```
s[0:3]     # numbers are treated as positions in slicing without .loc
```

```
doctor_id               1234
medication          DIAZEPAM
nb_beneficiaries           3
dtype: object
```

```
s.iloc[0:3]
```

```
doctor_id               1234
medication          DIAZEPAM
nb_beneficiaries           3
dtype: object
```

Subsetting Series (cont'd.)

- Subsetting Series can also be done using list of labels.

```
s[["doctor_id", "nb_beneficiaries"]]
```

```
doctor_id           1234
nb_beneficiaries       3
dtype: object
```

```
s.loc[["doctor_id", "nb_beneficiaries"]]
```

```
doctor_id           1234
nb_beneficiaries       3
dtype: object
```

```
s["doctor_id": "nb_beneficiaries"]
```

```
doctor_id              1234
medication          DIAZEPAM
nb_beneficiaries           3
dtype: object
```

included

```
s.loc["doctor_id": "nb_beneficiaries"]
```

```
doctor_id              1234
medication          DIAZEPAM
nb_beneficiaries           3
dtype: object
```

DataFrames

- It helps to think of DataFrames as spreadsheet-like tables made of a set of Series.
  - The Series share the same index.

	doctor_id	medication	nb_beneficiaries	spending
1	1952310666	CLONAZEPAM	226	$1,848.88
2	1952310666	DIAZEPAM	103	$662.87
3	1346358827	HYDROCODONE	331	$8,511.14
4	1548247315	MIRTAZAPINE	191	$3,131.96
5	1326175365	OXYCODONE HCL	87	$12,881.04
6	1518970284	DIGOXIN	54	$3,766.34

1	1952310666
2	1952310666
3	1346358827
4	1548247315
5	1326175365
6	1518970284

1	CLONAZEPAM
2	DIAZEPAM
3	HYDROCODONE
4	MIRTAZAPINE
5	OXYCODONE HCL
6	DIGOXIN

1	226
2	103
3	331
4	191
5	87
6	54

1	$1,848.88
2	$662.87
3	$8,511.14
4	$3,131.96
5	$12,881.04
6	$3,766.34

- The example above illustrates a collection of column Series as a DataFrame, but the analogy holds for row Series.

Reading a DataFrame from a file

- A DataFrame can be created by reading data from a file.
  - Pandas supports many input formats, including Excel, CSV, TSV, JSON, HDF5, etc.
- We can read the example comma-delimited (CSV) spending table using:

```
>>> spending_df  = pd.read_csv("data/spending.csv")
```

- Jupyter beautifies DataFrames by printing them as HTML tables.
  - Using `print` renders them as text-based output instead.
  - Try to avoid the explicit `print` by including the name as the last statement of the cell, or use `display`.

```
spending_df  = pd.read_csv("data/spending.csv")

spending_df
```

	unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
0	AB789982	1952310666	Psychiatry	CLONAZEPAM	226	1848.88
1	AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
2	CC128705	1298765423	Cardiology	NADOLOL	13	NaN
3	GH890091	1346358827	Family	HYDROCODONE	331	8511.14
4	YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1964.49
5	YY190561	1548247315	Psychiatry	GABAPENTIN	86	1807.16
6	YY572610	1548247315	Psychiatry	MIRTAZAPINE	191	3131.96
7	PL346720	1326175365	Family	OXYCODONE HCL	87	12881.04
8	GZ129032	1518970284	Hemato-oncology	DIGOXIN	54	3766.34

```
display(spending_df)
```

	unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
0	AB789982	1952310666	Psychiatry	CLONAZEPAM	226	1848.88
1	AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
2	CC128705	1298765423	Cardiology	NADOLOL	13	NaN
3	GH890091	1346358827	Family	HYDROCODONE	331	8511.14
4	YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1964.49
5	YY190561	1548247315	Psychiatry	GABAPENTIN	86	1807.16
6	YY572610	1548247315	Psychiatry	MIRTAZAPINE	191	3131.96
7	PL346720	1326175365	Family	OXYCODONE HCL	87	12881.04
8	GZ129032	1518970284	Hemato-oncology	DIGOXIN	54	3766.34

```
print(spending_df)
```

```
  unique_id   doctor_id        specialty     medication  nb_beneficiaries  \
0  AB789982  1952310666       Psychiatry     CLONAZEPAM               226
1  AV967778  1952310666       Psychiatry       DIAZEPAM               103
2  CC128705  1298765423       Cardiology        NADOLOL                13
3  GH890091  1346358827           Family    HYDROCODONE               331
4  YY219322  1548247315       Psychiatry     ALPRAZOLAM                28
5  YY190561  1548247315       Psychiatry     GABAPENTIN                86
6  YY572610  1548247315       Psychiatry    MIRTAZAPINE               191
7  PL346720  1326175365           Family  OXYCODONE HCL                87
8  GZ129032  1518970284  Hemato-oncology        DIGOXIN                54

   spending
0   1848.88
1    662.87
2       NaN
3   8511.14
4   1964.49
5   1807.16
6   3131.96
7  12881.04
8   3766.34
```

Dimensions of the DataFrame

- A useful property of the DataFrame is the `shape`.
  - This property describes the number of rows and the number of columns in your DataFrame.
- `shape` returns a tuple.
  - A list of elements describing the number of rows and the number of columns in an array.
  - Tuples are delimited by `()` rather than `[]`.

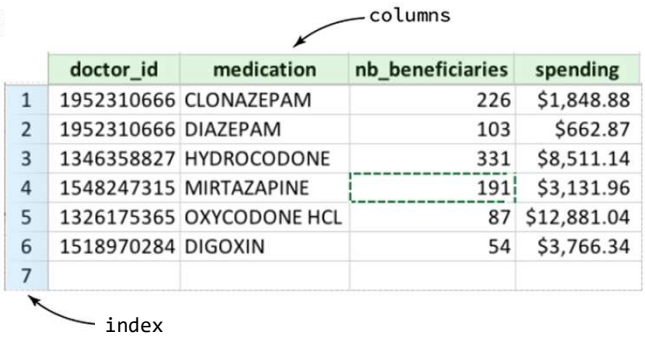
```
spending_df.shape
```

```
(9, 6)
```

- The above indicates that the `spending_df` DataFrame has 9 rows (table entries) and 6 columns.

DataFrame: `index` and `columns`

- As opposed to Series which only have an index, DataFrames have both rows and columns, each with their own labels (or names) and positions.
- A DataFrame can, therefore, be indexed by row label (`index`), row position, column names (`columns`), column position, or by a combination of row and column label/position.



	doctor_id	medication	nb_beneficiaries	spending
1	1952310666	CLONAZEPAM	226	$1,848.88
2	1952310666	DIAZEPAM	103	$662.87
3	1346358827	HYDROCODONE	331	$8,511.14
4	1548247315	MIRTAZAPINE	191	$3,131.96
5	1326175365	OXYCODONE HCL	87	$12,881.04
6	1518970284	DIGOXIN	54	$3,766.34
7				

DataFrame: `index` and `columns` (cont'd.)

- In the `spending` data example, since a row index was not explicitly provided, the labels for the rows are the same as the indices, i.e., 0 through 8.

	unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
0	AB789982	1952310666	Psychiatry	CLONAZEPAM	226	1848.88
1	AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
2	CC128705	1298765423	Cardiology	NADOLOL	13	NaN
3	GH890091	1346358827				
4	YY219322	1548247315	Psy			
5	YY190561	1548247315	Psy			
6	YY572610	1548247315	Psy			
7	PL346720	1326175365				
8	GZ129032	1518970284	Hemato-or			

```
spending_df.index
```

```
RangeIndex(start=0, stop=9, step=1)
```

```
spending_df.columns
```

```
Index(['unique_id', 'doctor_id', 'specialty', 'medication', 'nb_beneficiaries',
       'spending'],
      dtype='object')
```

Index location

- Row indexing can be carried out by passing the `iloc` (index location) operator a single index or a list of indexes.
  - The `iloc` operator uses [ ] instead of the ( ) that methods and functions use.
- When a single index is given, Pandas returns a Series.
  - Remember that a row is merely a Series.
- When a list of indices is given, Pandas returns another DataFrame.
  - The returned DataFrame is a subset of the original.

```
# Returns a row, or Series
spending_df.iloc[3]
```

```
unique_id              GH890091
doctor_id            1346358827
specialty                Family
medication          HYDROCODONE
nb_beneficiaries            331
spending                8511.14
Name: 3, dtype: object
```

```
# Returns two rows as a DataFrame
spending_df.iloc[[1,5]]
```

	unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
1	AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
5	YY190561	1548247315	Psychiatry	GABAPENTIN	86	1807.16

Indexing rows and columns with `iloc`

- You can pass `iloc` a combination of row and column indexes using the following construct:

  `dataframe.iloc[row_index_info, column_index_info]`

- `row_index_info` and `column_index_info` can be either a single integer or a list of integers.

```
spending_df.iloc[[2, 4], [1, 3]]
```

	doctor_id	medication
2	1298765423	NADOLOL
4	1548247315	ALPRAZOLAM

```
spending_df.iloc[3, 3]
```

```
'HYDROCODONE'
```

```
spending_df.iloc[2, [1, 3]]
```

```
doctor_id     1298765423
medication       NADOLOL
Name: 2, dtype: object
```

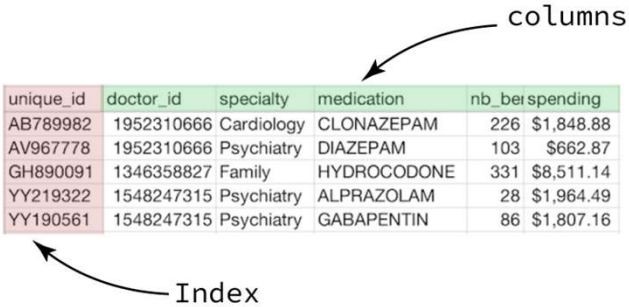
Reading tables with index labels

- Rather than using the default integer index label, a DataFrame can be indexed using one or more columns of data.
  - We will cover complex indexes (more than one column) later.

Specifying the index labels

- Specify the column name when reading the table.
  - E.g.: using the `read_csv()` function.
  - For example, we can use the `unique_id` column as index labels.

```
spending_df = pd.read_csv("data/spending.csv", index_col="unique_id")
```



unique_id	doctor_id	specialty	medication	nb_be	spending
AB789982	1952310666	Cardiology	CLONAZEPAM	226	$1,848.88
AV967778	1952310666	Psychiatry	DIAZEPAM	103	$662.87
GH890091	1346358827	Family	HYDROCODONE	331	$8,511.14
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	$1,964.49
YY190561	1548247315	Psychiatry	GABAPENTIN	86	$1,807.16

```
spending_df = pd.read_csv("data/spending.csv", index_col='unique_id')
spending_df
```

spending_df.index

unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
AB789982	1952310666	Psychiatry	CLONAZEPAM	226	1848.88
AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
CC128705	1298765423	Cardiology	NADOLOL	13	NaN
GH890091	1346358827	Family	HYDROCODONE	331	8511.14
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1964.49
YY190561	1548247315	Psychiatry	GABAPENTIN	86	1807.16
YY572610	1548247315	Psychiatry	MIRTAZAPINE	191	3131.96
PL346720	1326175365	Family	OXYCODONE HCL	87	12881.04
GZ129032	1518970284	Hemato-oncology	DIGOXIN	54	3766.34

```
spending_df_two_idx = pd.read_csv("data/spending.csv", index_col=['unique_id', "doctor_id"])
spending_df_two_idx
```

unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
AB789982	**1952310666**	Psychiatry	CLONAZEPAM	226	1848.88
AV967778	**1952310666**	Psychiatry	DIAZEPAM	103	662.87
CC128705	**1298765423**	Cardiology	NADOLOL	13	NaN
GH890091	**1346358827**	Family	HYDROCODONE	331	8511.14
YY219322	**1548247315**	Psychiatry	ALPRAZOLAM	28	1964.49
YY190561	**1548247315**	Psychiatry	GABAPENTIN	86	1807.16
YY572610	**1548247315**	Psychiatry	MIRTAZAPINE	191	3131.96
PL346720	**1326175365**	Family	OXYCODONE HCL	87	12881.04
GZ129032	**1518970284**	Hemato-oncology	DIGOXIN	54	3766.34

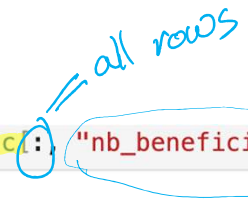
Indexing DataFrame columns using labels

- Accessing data in the DataFrame can also be done using indexes or column labels.
- Indexing columns takes a column's label or a list of column labels.
  - When a single label is given, Pandas returns a Series.
    - E.g., `spending_df["nb_beneficiaries"]`
    - Remember that a column is simply a Series.
  - When a list of labels is given, Pandas returns another DataFrame.
    - E.g., `spending_df[["doctor_id", "nb_beneficiaries"]]`
    - The returned DataFrame is a subset of the original.

```
spending_df["nb_beneficiaries"]
```

```
unique_id
AB789982    226
AV967778    103
CC128705     13
GH890091    331
YY219322     28
YY190561     86
YY572610    191
PL346720     87
GZ129032     54
Name: nb_beneficiaries, dtype: int64
```

```
spending_df.loc[:, "nb_beneficiaries"]
```




```
unique_id
AB789982    226
AV967778    103
CC128705     13
GH890091    331
YY219322     28
YY190561     86
YY572610    191
PL346720     87
GZ129032     54
Name: nb_beneficiaries, dtype: int64
```

```
spending_df[["doctor_id", "nb_beneficiaries"]]
```

unique_id	doctor_id	nb_beneficiaries
AB789982	1952310666	226
AV967778	1952310666	103
CC128705	1298765423	13
GH890091	1346358827	331
YY219322	1548247315	28
YY190561	1548247315	86
YY572610	1548247315	191
PL346720	1326175365	87
GZ129032	1518970284	54

```
spending_df.loc[:,["doctor_id", "nb_beneficiaries"]]
```

unique_id	doctor_id	nb_beneficiaries
AB789982	1952310666	226
AV967778	1952310666	103
CC128705	1298765423	13
GH890091	1346358827	331
YY219322	1548247315	28
YY190561	1548247315	86
YY572610	1548247315	191
PL346720	1326175365	87
GZ129032	1518970284	54

Indexing DataFrame rows using labels

- Row indexing using labels can be carried out using the `loc` (location) operator.
- `loc` can take a single label or a list of labels.
  - When a single label is given, Pandas returns a Series.
    - E.g., `spending_df.loc["AV967778"]`
    - Remember that a row is simply a Series.
  - When a list of labels is given, Pandas returns another DataFrame.
    - E.g., `spending_df.loc[["AV967778","YY219322"]]`
    - The returned DataFrame is a subset of the original.

```
spending_df.loc["AV967778"]
```

```
doctor_id           1952310666
specialty           Psychiatry
medication            DIAZEPAM
nb_beneficiaries           103
spending                662.87
Name: AV967778, dtype: object
```

```
spending_df.loc[["AV967778","YY219322"]]
```

unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1964.49

Subsetting by range on labels

- Although not useful or intuitive in this case, the range operator also works with the index and column labels.
- Therefore, the following lines are all valid lines.

```
spending_df.loc["AB789982", "specialty":"spending"]

spending_df.loc["AB789982":"GZ129032", "specialty"]

spending_df.loc["AB789982":"GZ129032", "specialty":"nb_beneficiaries"]
```

inclusive

} series.

- As opposed to ranges in lists, the upper limit of the range is included in the result.

```
spending_df.loc["AB789982":"GZ129032", "specialty":"nb_beneficiaries"]
```

unique_id	specialty	medication	nb_beneficiaries
AB789982	Psychiatry	CLONAZEPAM	226
AV967778	Psychiatry	DIAZEPAM	103
CC128705	Cardiology	NADOLOL	13
GH890091	Family	HYDROCODONE	331
YY219322	Psychiatry	ALPRAZOLAM	28
YY190561	Psychiatry	GABAPENTIN	86
YY572610	Psychiatry	MIRTAZAPINE	191
PL346720	Family	OXYCODONE HCL	87
GZ129032	Hemato-oncology	DIGOXIN	54

```
spending_df.loc[:, "specialty":"nb_beneficiaries"]
```

unique_id	specialty	medication	nb_beneficiaries
AB789982	Psychiatry	CLONAZEPAM	226
AV967778	Psychiatry	DIAZEPAM	103
CC128705	Cardiology	NADOLOL	13
GH890091	Family	HYDROCODONE	331
YY219322	Psychiatry	ALPRAZOLAM	28
YY190561	Psychiatry	GABAPENTIN	86
YY572610	Psychiatry	MIRTAZAPINE	191
PL346720	Family	OXYCODONE HCL	87
GZ129032	Hemato-oncology	DIGOXIN	54

DataFrame indexing summary

Syntax	Meaning
DataFrame.iloc[0, 1]	returns value at entry of index 0 and column 1
DataFrame.iloc[[0, 2]]	returns entries of indices 0 and 2
DataFrame.iloc[23, [0, 1]]	returns entry of index 23, and only values of columns 0 and 1
DataFrame.iloc[[1, 2], [0, 1]]	returns entries of indices 1 and 2, and only values of columns 0 and 1
DataFrame["col_name"]	Returns `Series` of "col_name"
DataFrame.loc[:, "col_name"]	Returns `Series` of "col_name"
DataFrame[["col_name_1", "col_name_2"]]	Returns `DataFrame` with "col_name_1" and "col_name_2"
DataFrame.loc[:, ["col_name_1", "col_name_2"]]	Returns `DataFrame` with "col_name_1" and "col_name_2"
DataFrame.loc["label"]	returns entry indexed by "label"
DataFrame.loc["label", ["col_3", 'col_5']]	returns entry indexed by "label", subsets columns to only "col_3" and "col_5"
DataFrame.loc[["label_1", "label_2"], ['col_3', 'col_5']]	returns entries with indices "label_1" and "label_2", subsets columns to only "col_3" and "col_5"

Inspecting DataFrames

- When a DataFrame contains a large number of rows it's common to use the method head or tail to display the first or the last five entries, respectively, of the DataFrame.
  - `df.head(n=10)` shows the top 10 lines of DataFrame `df`.
  - `df.tail(n=15)` shows the bottom 15 lines of DataFrame `df`.
  - By default, n = 5.

Inspecting DataFrames (cont'd.)

```
spending_df.head()
```

unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
AB789982	1952310666	Psychiatry	CLONAZEPAM	226	1848.88
AV967778	1952310666	Psychiatry	DIAZEPAM	103	662.87
CC128705	1298765423	Cardiology	NADOLOL	13	NaN
GH890091	1346358827	Family	HYDROCODONE	331	8511.14
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1964.49

Inspecting DataFrames (cont'd.)

```
spending_df.tail()
```

unique_id	doctor_id	specialty	medication	nb_beneficiaries	spending
YY219322	1548247315	Psychiatry	ALPRAZOLAM	28	1964.49
YY190561	1548247315	Psychiatry	GABAPENTIN	86	1807.16
YY572610	1548247315	Psychiatry	MIRTAZAPINE	191	3131.96
PL346720	1326175365	Family	OXYCODONE HCL	87	12881.04
GZ129032	1518970284	Hemato-oncology	DIGOXIN	54	3766.34